DevOps Exam App

Complete Setup & CI/CD Pipeline Guide

Jenkins • Docker • SonarQube • Trivy • Docker Scout • MySQL

This guide walks through every step needed to deploy the DevOps Exam App — from provisioning an Ubuntu VM to running a fully automated Jenkins CI/CD pipeline with security scanning, Docker image management, and live deployment via Docker Compose.

STEP 01 Launch Ubuntu VM

Ubuntu 24.04 | t2.large | 30 GB Storage

Security Group — Open Ports

Configure the following inbound rules in your cloud security group:

Type	Protocol	Port	Purpose
All TCP	TCP	0–65535	All TCP traffic
All ICMP - IPv4	ICMP	All	Ping / Diagnostics
SSH	TCP	22	Remote VM access
Custom TCP	TCP	3000	Application port
Custom TCP	TCP	8081	Service port
Custom TCP	TCP	8080	Jenkins dashboard
HTTPS	TCP	443	Secure web
Custom TCP	TCP	6443	Kubernetes API
HTTP	TCP	80	Web traffic
Custom TCP	TCP	9000	SonarQube console

STEP 02 Connect to VM

SSH into the instance and prepare the system

```
sudo su
```

```
sudo apt update
```

STEP 03 Install Jenkins

Create and run the Jenkins installation script

Create the script file:

```
vi jenkins.sh
```

jenkins.sh — script contents

```
#!/bin/bash

# Install OpenJDK 17 JRE Headless
sudo apt install openjdk-17-jre-headless -y

# Download Jenkins GPG key
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

# Add Jenkins repository
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt-get update
sudo apt-get install jenkins -y
```

Save & Execute

```
# In vi: press Esc, then type  :wq  and press Enter
sudo chmod +x jenkins.sh
./jenkins.sh
jenkins --version
```

> TIP *Access the Jenkins Dashboard at http://<VM-IP>:8080 once installation completes.*

STEP 04 Install Docker

Create and run the Docker installation script

```
vi docker.sh
```

docker.sh — script contents

```
#!/bin/bash
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.asc] \
```

```
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Save & Execute

```
sudo chmod +x docker.sh && ./docker.sh
docker --version
docker pull hello-world
```

> WARNING *If you see a permissions error when pulling images, run: sudo chmod 666 /var/run/docker.sock*

Post-Installation — Grant Jenkins Access to Docker

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo -u jenkins docker ps

sudo apt-get install -y python3-venv python3-pip
sudo apt-get install -y docker-compose-plugin
```

STEP 05 Install Trivy

Security scanner for filesystems and container images

```
vi trivy.sh
```

trivy.sh — script contents

```
#!/bin/bash
sudo apt-get install wget apt-transport-https gnupg
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
  | gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
  https://aquasecurity.github.io/trivy-repo/deb generic main" \
  | sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update
sudo apt-get install trivy
```

```
sudo chmod +x trivy.sh && ./trivy.sh
trivy --version
```

STEP 06 Install Docker Scout

Image vulnerability and recommendations tool

> WARNING *Log into DockerHub in your browser before running these commands.*

```
# Login from terminal
```

```
docker login -u <DockerHubUsername>
# Enter your DockerHub password when prompted

# Install Docker Scout CLI
curl -sSfL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh \
  | sh -s -- -b /usr/local/bin

sudo chmod 777 /var/run/docker.sock
```

STEP 07 Install SonarQube

Static code analysis — runs as a Docker container

```
docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
docker images
docker ps
```

> NOTE *Access SonarQube at http://<VM-IP>:9000 — Default credentials: admin / admin. You will be prompted to set a new password on first login.*

STEP 08 Jenkins Plugin Installation

Manage Jenkins → Plugins → Available Plugins

Search and install each of the following plugins:

Plugin	Plugin
SonarQube Scanner	Kubernetes
Docker	Kubernetes CLI
Docker Commons	Kubernetes Client API
Docker Pipeline	Kubernetes Credentials
Docker API	Kubernetes Credentials Provider
docker-build-step	Config File Provider
Pipeline Stage View	Prometheus Metrics
Email Extension Template	

> WARNING *The Prometheus Metrics plugin requires a Jenkins restart. Check 'Restart Jenkins when installation is complete' when prompted.*

STEP 09 Tools Configuration in Jenkins

Manage Jenkins → System Configuration → Tools

SonarQube Scanner Installation

Field	Value
Name	sonar-scanner
Install automatically	Yes (checked)
Version	7.1.0.4889 (latest)

Docker Installation

Field	Value
Name	docker
Install automatically	Yes (checked)
Installer type	Download from docker.com
Version	latest

Click Apply → Save when done.

STEP 10 Docker Hub Credentials

Manage Jenkins → Security → Credentials → Global → Add Credentials

Field	Value
Kind	Username with Password
Scope	Global
Username	<Your DockerHub Username>
Password	<Your DockerHub Password>
ID	docker
Description	docker

Click Create to save.

STEP 11 SonarQube System Configuration

Connect SonarQube to Jenkins — 5 sub-steps

11.1 Generate a SonarQube Token

In SonarQube console: Administration → Security → Users → Tokens (3-dash icon)

Field	Value
Name	token
Expires in	90 days

Click Generate and copy the token value.

11.2 Add Token to Jenkins Credentials

Manage Jenkins → Security → Credentials → Global → Add Credentials

Field	Value
Kind	Secret text
Scope	Global
Secret	<Paste the SonarQube token here>
ID	sonar-token
Description	sonar-token

11.3 Create a SonarQube Webhook

In SonarQube console: Administration → Configuration → Webhooks → Create

Field	Value
Name	jenkins
URL	http://<Jenkins-IP>:8080/sonarqube-webhook/

11.4 Configure SonarQube Server in Jenkins

Manage Jenkins → System Configuration → System → SonarQube Servers → Add SonarQube

Field	Value
Name	sonar
Server URL	http://<VM-IP>:9000
Authentication Token	Select sonar-token from dropdown

Click Apply → Save.

11.5 Generate Docker Registry Pipeline Syntax

Inside your Jenkins job: Pipeline Syntax → Sample step: withDockerRegistry

Field	Value
Docker registry URL	(leave empty for Docker Hub public registry)
Registry credentials	docker
Docker installation	docker

Click Generate Pipeline Syntax and paste the result into your pipeline script.

STEP 12 MySQL Database Access

Connect to the running MySQL container and run common queries

Connect to the Container

```
docker exec -it mysql_db mysql -u root -p
# Password: rootpass
```

Common SQL Operations

SQL Command	Description
SHOW DATABASES;	List all databases
USE devops_exam;	Switch to application database
SHOW TABLES;	List all tables
DESCRIBE results;	Show table structure
SELECT * FROM results;	View all exam results
SELECT * FROM results ORDER BY score DESC;	Results sorted by highest score
SELECT AVG(score) FROM results;	Calculate average score
SELECT COUNT(*) FROM results;	Count total submissions

STEP 13 Jenkins CI/CD Pipeline Script

Pipeline Flow

```
Git Checkout
↓  File System Scan  (Trivy)
↓  SonarQube Analysis
↓  Verify Docker Compose
↓  Build & Push Docker Image
↓  Docker Scout Image Analysis
↓  Deploy with Docker Compose
↓  Verify Deployment
```

Complete Jenkinsfile

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "kastrov/devopsexamapp:latest"
        SCANNER_HOME = tool 'sonar-scanner'
    }

    stages {

        stage('Git Checkout') {
            steps {
                git url: 'https://github.com/KastroVKiran/devops-exam-app.git',
                    branch: 'master'
            }
        }

        stage('File System Scan') {
            steps {
                sh "trivy fs --security-checks vuln,config --format table -o
trivy-fs-report.html ."
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonar') {
                    sh """
                    ${SCANNER_HOME}/bin/sonar-scanner \
                    -Dsonar.projectName=devops-exam-app \
                    -Dsonar.projectKey=devops-exam-app \
                    -Dsonar.sources=. \
                    -Dsonar.language=py \
                    -Dsonar.python.version=3 \
                    -Dsonar.host.url=http://localhost:9000
                    """
                }
            }
        }

        stage('Verify Docker Compose') {
            steps {
                sh '''
                docker compose version || { echo "Docker Compose not available"; exit 1; }
                '''
            }
        }

        stage('Build Docker Image') {
```

```
            steps {
                dir('backend') {
                    script {
                        withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                            sh "docker build -t ${DOCKER_IMAGE} ."
                            sh "docker push ${DOCKER_IMAGE}"
                        }
                    }
                }
            }
        }

        stage('Docker Scout Image Analysis') {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                        sh "docker-scout quickview ${DOCKER_IMAGE}"
                        sh "docker-scout cves ${DOCKER_IMAGE}"
                        sh "docker-scout recommendations ${DOCKER_IMAGE}"
                    }
                }
            }
        }

        stage('Deploy with Docker Compose') {
            steps {
                sh '''
                docker compose down --remove-orphans || true
                docker compose up -d --build
                echo "Waiting for MySQL to be ready..."
                timeout 120s bash -c '
                while ! docker compose exec -T mysql mysqladmin ping -uroot -prootpass
--silent;
                do
                    sleep 5;
                    docker compose logs mysql --tail=5 || true;
                done'
                sleep 10
                '''
            }
        }

        stage('Verify Deployment') {
            steps {
                sh '''
                echo "=== Container Status ==="
                docker compose ps -a
                echo "=== Testing Flask Endpoint ==="
                curl -I http://localhost:5000 || true
                '''
            }
        }
    }

    post {
        success {
            echo 'Deployment successful!'
            sh 'docker compose ps'
            archiveArtifacts artifacts: 'trivy-fs-report.html', allowEmptyArchive: true
        }
        failure {
            echo 'Pipeline failed. Check logs above.'
            sh '''
            echo "=== Error Investigation ==="
            docker compose logs --tail=50 || true
            '''
        }
        always {
```

```
            sh '''
            echo "=== Final Logs ==="
            docker compose logs --tail=20 || true
            '''
            archiveArtifacts artifacts: 'trivy-fs-report.html', allowEmptyArchive: true
        }
    }
}
```

DevOps Exam App — Setup Complete

Jenkins • Docker • SonarQube • Trivy • Docker Scout • MySQL